GUÍA DE LABORATORIO 09

Servicios REST y operaciones CRUD en Angular

Desarrollo de un portal académico conectado a una API simulada

En este laboratorio conectarás el portal académico con un servidor HTTP local. Consultarás estudiantes, registrarás nuevos datos, editarás registros y confirmarás su eliminación. Los cambios se conservarán en un archivo del servidor, incluso después de recargar el navegador.

La práctica explica el contrato de la API, los servicios de Angular y el comportamiento de los Observables antes de construir cada parte. Al finalizar tendrás una aplicación con estados de carga y error, formularios validados y pruebas que comprueban las solicitudes HTTP.

El conjunto de datos es ficticio. Las categorías académicas del ejemplo son reglas didácticas y no representan normas de evaluación institucionales.

Ruta de lectura

Sigue las secciones en orden. Cada archivo se presenta completo y los puntos de comprobación te permiten detectar un problema antes de avanzar. En Word puedes usar el panel de navegación o estos enlaces para localizar una actividad.

1 Resultados y organización del laboratorio

2 Conceptos de HTTP y REST

3 Contrato de la API académica

4 Observables y estados de la interfaz

5 Preparación del proyecto

6 Construcción de la API simulada

7 Modelos y servicio de estudiantes

8 Configuración y navegación de Angular

9 Estructura visual y componentes

10 Consulta y eliminación de estudiantes

11 Registro y edición de estudiantes

12 Resumen académico desde la API

13 Pruebas automatizadas del portal

14 Pruebas manuales y persistencia

15 Depuración y errores frecuentes

16 Reto de aplicación individual

17 Evidencias y criterios de evaluación

18 Preguntas de comprobación

19 Referencias oficiales

1 Resultados y organización del laboratorio

La semana 9 desarrolla servicios REST en Angular, después del trabajo con rutas, componentes, pipes y formularios de la semana 8. El portal conserva esas piezas y añade una capa de comunicación HTTP. El navegador deja de ser el único lugar que mantiene la lista de estudiantes.

Logro propuesto del laboratorio. Al finalizar, implementarás un servicio tipado para consumir una API REST simulada y lo integrarás con vistas de consulta, registro, edición y eliminación, gestionando carga, errores, validación y verificación de las operaciones.

Conocimientos previos

1. Tipos, interfaces, uniones y genéricos básicos de TypeScript; importación de módulos.
2. Componentes standalone, inyección con `inject()`, signals y valores `computed()`.
3. Rutas, `RouterLink`, `@if`, `@for` y seguimiento de elementos por identificador.
4. Formularios reactivos tipados, validadores y ejecución de scripts npm.

Resultados observables

Resultado	Evidencia que debes mostrar
Comprender el contrato HTTP	Identificas método, URL, cuerpo y respuesta de cada operación.
Separar responsabilidades	Las páginas usan EstudiantesApi; la tarjeta solo presenta datos y emite eventos.
Implementar CRUD	GET consulta, POST crea, PATCH edita y DELETE elimina con confirmación.
Diferenciar PUT y PATCH	Pruebas una representación completa y un cambio parcial.
Gestionar asincronía	La interfaz informa carga, vacío, éxito y error sin perder datos ante un fallo.
Comprobar persistencia	Un registro sigue disponible tras recargar y reiniciar la API.
Verificar el código	La compilación estricta y las 19 pruebas automatizadas terminan correctamente.

Distribución del tiempo

La distribución es orientativa. Antes de la sesión, prepara el proyecto y los archivos estructurales de las secciones 5, 8 y 9. Así el tiempo presencial puede centrarse en el servicio HTTP, el comportamiento de las vistas y la comprobación de resultados. Si empiezas desde cero, completa esa preparación previa con los archivos de esta guía.

Actividad
Contrato HTTP, recursos y Observables
API local y revisión de los datos iniciales
Servicio tipado y configuración de HttpClient

Actividad
Lectura de estudiantes y gestión de estados
Registro, edición y validación del formulario
Confirmación de eliminación y persistencia
Pruebas automatizadas y manuales
Reto, evidencias y reflexión

2 Conceptos de HTTP y REST

Cliente servidor y recurso

El cliente es la aplicación Angular que ejecuta el navegador. El servidor es el proceso que recibe solicitudes HTTP y responde. En este laboratorio JSON Server cumple ese papel: expone el recurso `estudiantes` a partir de un archivo JSON. JSON describe los datos intercambiados; no es un servidor ni un lenguaje para implementar un backend.

Una API es una interfaz que define cómo otro programa puede solicitar una operación. Un endpoint combina una dirección y el uso esperado de un método HTTP. Por ejemplo, `GET /estudiantes/101` consulta el estudiante cuyo identificador es la cadena `101`; `DELETE` sobre la misma dirección solicita su eliminación.

REST es un estilo arquitectónico que organiza la interacción alrededor de recursos y representaciones. Una API con JSON no cumple automáticamente todas las restricciones de REST. Aquí se trabaja un patrón habitual de API REST para comprender recursos, métodos y respuestas. La ausencia de estado de sesión entre solicitudes no significa que el servidor no conserve datos.

Partes de una solicitud y una respuesta

Parte	Ejemplo	Qué expresa
Método	POST	La operación solicitada.
URL	http://127.0.0.1:3000/estudiantes	El recurso y el servidor de destino.
Cabecera	Content-Type: application/json	El formato del cuerpo enviado.
Cuerpo	Nombre, programa, promedio y asistencia	La representación o los cambios solicitados.
Estado de respuesta	201 Created	El resultado HTTP de la operación.
Cuerpo de respuesta	Estudiante creado con su id	Los datos que devuelve el servidor.

La URL absoluta contiene esquema, host, puerto y ruta. `127.0.0.1` apunta al equipo local; `3000` identifica el puerto de la API. El frontend se sirve normalmente desde `http://localhost:4200`. Una ruta de Angular, como `/registro/101`, muestra una página del portal; una ruta de la API, como `/estudiantes/101`, devuelve datos JSON.

CRUD y los métodos HTTP

CRUD resume crear, leer, actualizar y eliminar. HTTP dispone de métodos con semánticas distintas. GET consulta; POST suele crear un recurso dentro de una colección; PUT envía la representación completa para reemplazar el recurso identificado; PATCH aplica modificaciones parciales; DELETE solicita eliminarlo. El contrato de cada API determina los detalles y los campos admitidos. [R1]

PUT se considera idempotente: repetir la misma solicitud busca el mismo efecto final. GET y DELETE también tienen semántica idempotente, aunque una segunda eliminación puede devolver un estado distinto. POST no tiene esa garantía en general. PATCH puede ser idempotente o no según el tipo de modificación. Asignar `asistencia: 95` es distinto de una operación que incrementa un contador.

Al editar desde el formulario usaremos PATCH aunque enviemos los cuatro campos editables. El servicio también ofrecerá PUT para estudiar el reemplazo y verificarlo en las pruebas; la interfaz no necesita un segundo formulario para ello.

CORS y el origen de la aplicación

El origen combina esquema, host y puerto. El navegador considera distintos `localhost:4200` y `127.0.0.1:3000`; por eso una solicitud entre ellos necesita autorización CORS del servidor. JSON Server proporciona esa autorización en este entorno local. No se resuelve un problema CORS desactivando la seguridad del navegador. [R2]

El navegador puede enviar una solicitud OPTIONS de comprobación previa, llamada preflight, antes de POST con JSON, PATCH o DELETE. Ver OPTIONS y luego PATCH en Network no significa que se haya editado dos veces. CORS es una política del navegador; no es autenticación ni autorización de usuarios.

3 Contrato de la API académica

La colección contiene objetos con cinco propiedades. Los promedios están entre 0 y 20; la asistencia se expresa como un porcentaje entre 0 y 100. El campo `programa` admite `Software` o `Sistemas`. Estos límites se comprobarán en el formulario; el simulador no impone las reglas del negocio.

Campo	Tipo	Regla del ejemplo
id	string	Lo asigna el servidor al crear; el cliente no lo fabrica.
nombre	string	Entre 3 y 80 caracteres tras quitar espacios exteriores.
programa	Software o Sistemas	Una de las dos opciones del formulario.
promedio	number	Número finito entre 0 y 20; cero es válido.
asistencia	number	Número finito entre 0 y 100; cero es válido.

JSON Server 1 utiliza identificadores de tipo cadena. Los registros iniciales tendrán valores como `"101"`; una creación puede recibir un identificador opaco diferente. No supongas que el siguiente será `"107"` ni lo conviertas a número. Utiliza el id que devuelve POST para editar o eliminar. [R3]

Método	Ruta relativa	Cuerpo enviado	Resultado esperado
GET	/estudiantes	Sin cuerpo	200 con un arreglo, que puede estar vacío.

Método	Ruta relativa	Cuerpo enviado	Resultado esperado
GET	/estudiantes/:id	Sin cuerpo	200 con un objeto; 404 si no existe.
POST	/estudiantes	Los cuatro campos editables	201 con el estudiante y su id.
PATCH	/estudiantes/:id	Uno o más campos editables	200 con el registro actualizado.
PUT	/estudiantes/:id	Los cuatro campos editables	200 con la representación reemplazada.
DELETE	/estudiantes/:id	Sin cuerpo	200 en el simulador utilizado.

En esta tabla `:id` representa un segmento variable de la URL, no texto que debas enviar literalmente. Otras APIs pueden responder 204 a DELETE, sin cuerpo. Nuestro servicio no depende del contenido de esa respuesta: al recibir éxito actualiza la lista local. La prueba automatizada también comprueba una respuesta sin datos.

Estados que debes interpretar

Estado	Significado	Respuesta de la interfaz
200	Operación correcta	Presentar los datos o confirmar el cambio.
201	Recurso creado	Mostrar el nombre y el id devueltos.
204	Éxito sin cuerpo en algunas APIs	No intentar leer campos del cuerpo.
404	Recurso no encontrado	Informar que el registro ya no existe.
5xx	Fallo del servidor	Mostrar un mensaje y conservar los datos locales.
0 en Angular	Fallo de red o acceso bloqueado	Revisar servidor, URL y CORS; no es un estado HTTP enviado por el servidor.

Un arreglo vacío con 200 es una consulta correcta sin registros. Un error 500 es una consulta fallida. No conviertas todos los errores en `[]`, porque ocultarías el problema y la pantalla parecería vacía cuando la API no respondió correctamente.

4 Observables y estados de la interfaz

Servicio inyección y HttpClient

Un servicio reúne operaciones que pueden compartir varias páginas. `EstudiantesApi` conocerá la URL y construirá las solicitudes; las páginas decidirán qué mostrar. `@Injectable({ providedIn: 'root' })` registra el servicio en el inyector raíz. `inject(EstudiantesApi)` solicita una instancia gestionada por Angular. [R4]

`HttpClient` es el cliente HTTP de Angular. Declararemos `provideHttpClient()` en la configuración para hacer explícita su provisión. En Angular 22 utiliza Fetch de forma predeterminada. No se requiere importar `HttpClientModule` ni crear un NgModule para este proyecto standalone. [R5]

Observable y suscripción

Un Observable<T> representa una secuencia de valores que puede llegar de forma asíncrona. Los Observables de HttpClient son fríos: construir api.listar() no envía todavía el GET; la suscripción inicia la solicitud. Suscribirse dos veces al mismo Observable HTTP inicia dos solicitudes independientes. [R6]

Al usar subscribe({ next, error }), next recibe el cuerpo de una respuesta correcta y error recibe el fallo. Las solicitudes normales de HttpClient emiten la respuesta y terminan. No devuelvas la suscripción desde el servicio como si fuera el dato: devuelve el Observable y deja que la página gestione el resultado.

La anotación http.get<Estudiante[]>() ayuda al compilador, pero no verifica en ejecución el JSON recibido. En la práctica controlamos los datos del simulador. Una aplicación conectada a un servidor externo debería validar el contrato de la respuesta antes de tratar datos desconocidos como estudiantes. [R6]

Operadores empleados en esta práctica

Operador o API	Función	Uso concreto
finalize	Ejecuta una acción al terminar o cancelar la suscripción.	Restablecer cargando, guardando o eliminando incluso si hubo error.
takeUntilDestroyed	Completa al destruirse el contexto indicado.	Cerrar suscripciones de la página usando su DestroyRef.
switchMap	Cambia al Observable más reciente y cancela la suscripción anterior.	Consultar el id actual al cambiar los parámetros de edición.
catchError	Intercepta un fallo y devuelve otro Observable.	Mostrar error de carga y emitir null dentro de la consulta de edición.
tap	Realiza efectos sin transformar el valor.	Preparar el formulario cuando cambia el parámetro de ruta.
of	Emite el valor indicado y termina.	Representar que un registro nuevo no necesita un GET de detalle.

Usamos takeUntilDestroyed(this.destroyRef) con un DestroyRef inyectado porque los métodos se ejecutan fuera del contexto de inyección inicial. Al salir de una página se cancela su suscripción HTTP pendiente. Cancelar en el cliente no garantiza deshacer un cambio que el servidor ya haya aplicado. [R7]

En el formulario, catchError se sitúa dentro del Observable que devuelve switchMap. Así un 404 de una consulta puede mostrarse sin terminar toda la escucha de parámetros. switchMap resulta apropiado para sustituir lecturas anteriores; no se usa aquí para lanzar y cancelar guardados sucesivos.

Estados de carga y modificación

Cada lectura distingue carga, error, resultado vacío y resultado con datos. Las modificaciones usan estados propios: mientras se guarda o elimina se deshabilita la acción correspondiente. Un segundo intento durante la misma operación se bloquea también en el método TypeScript, no solo en el botón.

La tarjeta se retira de la lista después de que DELETE responde correctamente. Si el servidor falla, la tarjeta permanece y se muestra el error. El formulario conserva sus valores ante un fallo de guardado.

Evitamos repetir POST automáticamente: si la respuesta se perdió pero el servidor creó el registro, reintentar sin comprobarlo puede producir duplicados.

5 Preparación del proyecto

Versiones y herramientas

Se mantiene la línea Angular 22.1 del portal anterior y se fijan versiones compatibles para reproducir la práctica. No es necesario instalar las versiones globales de Angular CLI. Node.js 24.19.0 pertenece a la línea LTS utilizada en la verificación; Angular 22 admite Node.js 24 desde 24.15.0 y TypeScript 6.0.x. [R8]

Herramienta	Versión usada	Propósito
Node.js y npm	24.19.0 y 11.9.0	Ejecutar CLI, servidor local y scripts.
Angular	22.1.7	Componentes, rutas, formularios y HttpClient.
Angular CLI y build	22.1.8	Generar, servir, compilar y ejecutar pruebas.
TypeScript	6.0.3	Comprobación estática y tipado estricto.
RxJS	7.8.2	Observables y operadores.
Bootstrap	5.3.8	Estilos y cuadrícula adaptable.
JSON Server	1.0.0-beta.15	API simulada local con un archivo de datos.
Vitest y jsdom	4.1.11 y 28.1.0	Pruebas unitarias y DOM simulado.
Prettier	3.8.1	Formato opcional del código.

JSON Server está en una versión previa de la línea 1. Se utiliza exclusivamente como simulador local y se fija su versión porque esa línea puede incorporar cambios incompatibles. No ofrece por sí sola un backend de producción con autenticación y validación del negocio. [R3]

Crear una copia de trabajo nueva

1. Abre Visual Studio Code y una terminal en la carpeta que destinarás al laboratorio. Comprueba `node --version` y `npm --version`.
2. Crea un proyecto nuevo llamado portal-rest. Así conservas el trabajo de la semana 8 y puedes seguir esta guía desde cero.
3. Entra en la carpeta generada. Los comandos posteriores se ejecutan desde esa raíz, donde se encuentra package.json.
4. Reemplaza package.json, angular.json y tsconfig.json con los contenidos completos siguientes. Conserva tsconfig.app.json, tsconfig.spec.json, public/favicon.ico y los demás archivos de configuración generados por CLI.
5. Crea las carpetas y archivos que indica el inventario; elimina la plantilla inicial solo al reemplazarla por los archivos de esta guía.

Comando en la raíz del proyecto

```
npx --yes @angular/cli@22.1.8 new portal-rest --standalone --routing --style=css --skip-git --skip-install --ssr=false --defaults
cd portal-rest
```

El comando genera una aplicación standalone con rutas y CSS, sin renderizado del servidor. --skip-install permite fijar las dependencias antes de instalarlas. Si la carpeta portal-rest ya existe, elige otra carpeta de trabajo vacía y mantén el nombre del proyecto; evita ejecutar el generador sobre un proyecto existente.

Los scripts separan el servidor de Angular, la API y las pruebas. Las versiones sin ^ ni ~ mantienen las dependencias directas fijadas. packageManager documenta la versión de npm utilizada; no instala esa versión automáticamente.

package.json Parte 1 de 3

```
{
  "name": "portal-rest",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "watch": "ng build --watch --configuration development",
    "test": "ng test",
    "test:ci": "ng test --watch=false",
    "verify": "ng build && ng test --watch=false",
    "api": "json-server api/db.json --port 3000 --host 127.0.0.1",
    "api:reset": "node scripts/reset-api.mjs"
```

package.json Parte 2 de 3

```
  },
  "private": true,
  "packageManager": "npm@11.9.0",
  "dependencies": {
    "@angular/common": "22.1.7",
    "@angular/compiler": "22.1.7",
    "@angular/core": "22.1.7",
    "@angular/forms": "22.1.7",
    "@angular/platform-browser": "22.1.7",
    "@angular/router": "22.1.7",
    "bootstrap": "5.3.8",
    "rxjs": "7.8.2",
    "tslib": "2.8.1"
```

package.json Parte 3 de 3

```
  },
  "devDependencies": {
    "@angular/build": "22.1.8",
    "@angular/cli": "22.1.8",
    "@angular/compiler-cli": "22.1.7",
    "jsdom": "28.1.0",
    "prettier": "3.8.1",
    "typescript": "6.0.3",
    "vitest": "4.1.11",
    "json-server": "1.0.0-beta.15"
  }
}
```

La configuración conserva los builders actuales de Angular CLI. El presupuesto inicial de advertencia se fija en 600 kB para este ejemplo que incluye Bootstrap; no sustituye la evaluación de rendimiento ni una optimización de producción.

angular.json Parte 1 de 5

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "cli": {
    "packageManager": "npm"
  },
  "newProjectRoot": "projects",
  "projects": {
    "portal-rest": {
      "projectType": "application",
      "schematics": {},
      "root": "",
      "sourceRoot": "src",
      "prefix": "app",
      "architect": {
```

angular.json Parte 2 de 5

```
        "build": {
          "builder": "@angular/build:application",
          "options": {
            "browser": "src/main.ts",
            "tsConfig": "tsconfig.app.json",
            "assets": [
              {
                "glob": "**/*",
                "input": "public"
              }
            ],
            "styles": ["src/styles.css"]
          },
          "configurations": {
            "production": {
```

angular.json Parte 3 de 5

```
              "budgets": [
                {
                  "type": "initial",
                  "maximumWarning": "600kB",
                  "maximumError": "1MB"
                },
                {
                  "type": "anyComponentStyle",
                  "maximumWarning": "4kB",
                  "maximumError": "8kB"
                }
              ],
              "outputHashing": "all"
            },
            "development": {
```

angular.json Parte 4 de 5

```
            "optimization": false,
            "extractLicenses": false,
            "sourceMap": true
          }
        },
        "defaultConfiguration": "production"
      },
      "serve": {
        "builder": "@angular/build:dev-server",
        "configurations": {
          "production": {
            "buildTarget": "portal-rest:build:production"
          },
          "development": {
            "buildTarget": "portal-rest:build:development"
```

angular.json Parte 5 de 5

```
          }
        },
        "defaultConfiguration": "development"
      },
      "test": {
        "builder": "@angular/build:unit-test"
      }
    }
  }
}
```

El modo estricto y strictTemplates ayudan a detectar tipos incompatibles en código y plantillas. Los dos tsconfig referenciados son los creados por CLI; no los elimines.

tsconfig.json Parte 1 de 2

```
{
  "compileOnSave": false,
  "compilerOptions": {
    "strict": true,
    "noImplicitOverride": true,
    "noPropertyAccessFromIndexSignature": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "skipLibCheck": true,
    "isolatedModules": true,
    "experimentalDecorators": true,
    "importHelpers": true,
    "target": "ES2022",
    "module": "preserve"
  },
  "angularCompilerOptions": {
```

tsconfig.json Parte 2 de 2

```
    "strictTemplates": true,
    "enableI18nLegacyMessageIdFormat": false,
    "strictInjectionParameters": true,
    "strictInputAccessModifiers": true
  },
  "files": [],
  "references": [
    {
      "path": "./tsconfig.app.json"
    },
    {
      "path": "./tsconfig.spec.json"
    }
  ]
}
```

Comando en la raíz del proyecto

```
npm install
```

La instalación crea package-lock.json. Consérvalo y entrégalo: registra la resolución de dependencias transitivas. En una copia que ya incluya ese archivo coherente con package.json puedes usar `npm ci`. No uses `--force` para ocultar una incompatibilidad de versiones.

Inventario de archivos

Carpeta o archivo	Responsabilidad
api/seed.json y api/db.json	Datos iniciales y archivo de datos que modifica la API.
scripts/reset-api.mjs	Restaurar los seis registros iniciales cuando lo solicites.
src/main.ts y src/index.html	Arranque de Angular y documento HTML.
src/styles.css	Bootstrap y estilos globales.
src/app/app.ts, app.html y app.css	Estructura principal del portal.
src/app/app.config.ts y app.routes.ts	Proveedores, locale y navegación.
src/app/modelos/estudiante.ts	Contrato de datos y reglas didácticas.
src/app/servicios	Cliente de la API y traducción de errores HTTP.
src/app/componentes	Tarjeta de estudiante y su plantilla.
src/app/pipes	Presentación del estado académico.
src/app/paginas	Resumen, listado, registro y página no encontrada.
Archivos con extensión .spec.ts	Las tres suites de pruebas del laboratorio.

En el portal de esta semana no se utiliza el almacén en memoria ni el arreglo local de estudiantes de la semana 8. Los únicos datos iniciales se encuentran en api/seed.json. Si adaptas tu proyecto anterior, retira las referencias al store y cambia el tipo de id a string en todos los consumidores.

6 Construcción de la API simulada

Preparar los datos y el reinicio explícito

Crea api/seed.json con los seis registros ficticios. JSON exige nombres de propiedad y cadenas entre comillas dobles; no admite comentarios ni comas finales. El punto se usa como separador decimal en JSON, aunque la interfaz muestre números con formato peruano.

La propiedad estudiantes define una colección. Cada objeto dispone de un id de cadena. El archivo seed contiene la copia inicial; no será el archivo que escriba JSON Server.

api/seed.json Parte 1 de 3

```
{
  "estudiantes": [
    {
      "id": "101",
      "nombre": "Ana Torres",
      "programa": "Software",
      "promedio": 17.2,
      "asistencia": 94
    },
    {
      "id": "102",
      "nombre": "Luis Ramírez",
      "programa": "Sistemas",
      "promedio": 10.4,
      "asistencia": 72
    },
```

api/seed.json Parte 2 de 3

```
    {
      "id": "103",
      "nombre": "María Paredes",
      "programa": "Software",
      "promedio": 13.5,
      "asistencia": 88
    },
    {
      "id": "104",
      "nombre": "Carlos Vega",
      "programa": "Sistemas",
      "promedio": 8.9,
      "asistencia": 66
    },
    {
      "id": "105",
```

api/seed.json Parte 3 de 3

```
    "nombre": "Sofía Medina",
    "programa": "Software",
    "promedio": 18.1,
    "asistencia": 97
   },
   {
    "id": "106",
    "nombre": "Diego Salas",
    "programa": "Sistemas",
    "promedio": 11.8,
    "asistencia": 81
   }
 ]
}
```

Crea también la carpeta scripts. Este script copia la semilla a api/db.json usando rutas relativas a su propio archivo. Solo lo ejecutarás para inicializar o restaurar los datos, con la API detenida.

scripts/reset-api.mjs

```
import { copyFile } from 'node:fs/promises';

await copyFile(
  new URL('../api/seed.json', import.meta.url),
  new URL('../api/db.json', import.meta.url),
);
console.log('Se restauraron los seis registros ficticios iniciales.');
```

Comprueba el avance. Después del comando de inicialización existirá api/db.json con los mismos seis registros.

Comando en la raíz del proyecto

```
npm run api:reset
```

Resultado esperado: “Se restauraron los seis registros ficticios iniciales”. Ejecutar este comando reemplaza los datos actuales del laboratorio; no lo incluyas como paso automático antes de cada arranque. Los registros creados permanecerán al reiniciar la API si conservas db.json.

Iniciar y consultar el servidor

1. En una terminal A, situada en portal-rest, inicia la API con el comando siguiente y deja esa terminal abierta.
2. Abre http://127.0.0.1:3000/estudiantes en el navegador. Debes recibir un arreglo JSON de seis objetos.
3. Consulta http://127.0.0.1:3000/estudiantes/101. Debe aparecer Ana Torres y un solo objeto, no un arreglo.
4. Consulta /estudiantes/inexistente en esa misma API para observar una respuesta 404. Volverás a probar ese caso desde Angular.
5. Cuando necesites detener el servidor, usa Ctrl+C en la terminal A. Para iniciarlo de nuevo ejecuta npm run api, sin restaurar la semilla.

Comando en la raíz del proyecto

```
npm run api
```

Si el puerto 3000 está ocupado, detén tu proceso anterior o utiliza otro puerto y actualiza API_URL para que ambos coincidan. El nombre de la ruta depende de la propiedad estudiantes del JSON. No cambies esa propiedad sin cambiar también el servicio Angular.

7 Modelos y servicio de estudiantes

Modelo de datos

Una interfaz define la forma de un estudiante durante la compilación. readonly evita reasignaciones de propiedades a través de esa referencia tipada; no congela objetos en ejecución. Omit<Estudiante, 'id'> describe los datos enviados al crear porque el servidor asigna el identificador.

Las categorías Destacado, Aprobado y En riesgo se conservan como regla didáctica del portal. El umbral de 14 y el de 11 sirven al ejemplo y no establecen políticas académicas oficiales.

src/app/modelos/estudiante.ts

```
export type Programa = 'Software' | 'Sistemas';
export type EstadoAcademico = 'Destacado' | 'Aprobado' | 'En riesgo';

export interface Estudiante {
  readonly id: string;
  readonly nombre: string;
  readonly programa: Programa;
  readonly promedio: number;
  readonly asistencia: number;
}

export type BorradorEstudiante = Omit<Estudiante, 'id'>;

export function estadoAcademico(promedio: number): EstadoAcademico {
  if (promedio >= 14) return 'Destacado';
  return promedio >= 11 ? 'Aprobado' : 'En riesgo';
}
```

Servicio de acceso a la API

Crea servicios/estudiantes-api.ts. Cada método devuelve un Observable y concentra un tipo de solicitud. El servicio no muestra alertas ni modifica signals de las páginas; así puede reutilizarse y probarse sin conocer la interfaz.

Partial<BorradorEstudiante> permite enviar un subconjunto de campos con PATCH. PUT solicita la representación completa. encodeURIComponent() protege el segmento del id al formar la URL. HttpClient serializa los objetos enviados como JSON.

src/app/servicios/estudiantes-api.ts Parte 1 de 3

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import type { BorradorEstudiante, Estudiante } from '../modelos/estudiante';

export const API_URL = 'http://127.0.0.1:3000/estudiantes';

@Injectable({ providedIn: 'root' })
export class EstudiantesApi {
  private readonly http = inject(HttpClient);

  listar(): Observable<Estudiante[]> {
```

src/app/servicios/estudiantes-api.ts Parte 2 de 3

```
  return this.http.get<Estudiante[]>(API_URL);
 }
 obtener(id: string): Observable<Estudiante> {
  return this.http.get<Estudiante>(this.url(id));
 }
 crear(datos: BorradorEstudiante): Observable<Estudiante> {
  return this.http.post<Estudiante>(API_URL, datos);
 }
 actualizar(
  id: string,
  cambios: Partial<BorradorEstudiante>,
 ): Observable<Estudiante> {
```

src/app/servicios/estudiantes-api.ts Parte 3 de 3

```
  return this.http.patch<Estudiante>(this.url(id), cambios);
 }
 reemplazar(id: string, datos: BorradorEstudiante): Observable<Estudiante> {
  return this.http.put<Estudiante>(this.url(id), datos);
 }
 eliminar(id: string): Observable<unknown> {
  return this.http.delete<unknown>(this.url(id));
 }
 private url(id: string): string {
  return `${API_URL}/${encodeURIComponent(id)}`;
 }
}
```

Comprueba el avance. listar devuelve un Observable de un arreglo; obtener, crear, actualizar y reemplazar devuelven uno de un estudiante. Ningún método fabrica un id.

Mensajes de error

El tipo unknown obliga a inspeccionar el error antes de acceder a sus propiedades. El ayudante reconoce HttpErrorResponse y produce mensajes comprensibles; conserva una categoría genérica para un error inesperado. La consola y Network siguen siendo necesarias para diagnosticar la causa técnica.

Un estado 0 no permite afirmar por sí solo si la causa es CORS, un servidor detenido o un fallo de red. El mensaje pide comprobar las tres posibilidades. Las vistas decidirán dónde y cuándo mostrarlo.

src/app/servicios/mensaje-http.ts

```
import { HttpErrorResponse } from '@angular/common/http';

export function mensajeHttp(error: unknown): string {
 if (!(error instanceof HttpErrorResponse)) {
  return 'Ocurrió un error inesperado. Revisa la consola de desarrollo.';
 }
 if (error.status === 0) {
  return 'No se pudo contactar la API. Comprueba el servidor, la URL y CORS.';
 }
 if (error.status === 404) return 'El registro solicitado ya no existe.';
 if (error.status >= 500)
  return 'La API falló. Intenta la operación más tarde.';
 return `La API rechazó la solicitud con estado ${error.status}.`;
}
```

8 Configuración y navegación de Angular

Proveedores globales y locale

La configuración activa el router, hace explícito HttpClient y registra es-PE para los pipes numéricos y de fecha. La provisión global del cliente HTTP evita declarar un cliente diferente en cada página. Los componentes standalone importan directamente las directivas y pipes que usan en sus plantillas.

registerLocaleData() incorpora los datos de presentación y LOCALE_ID selecciona el locale. Configurar es-PE no cambia el formato numérico del JSON intercambiado con la API.

src/app/app.config.ts Parte 1 de 2

```
import { registerLocaleData } from '@angular/common';
import { provideHttpClient } from '@angular/common/http';
import localeEsPe from '@angular/common/locales/es-PE';
import {
  ApplicationConfig,
  LOCALE_ID,
  provideBrowserGlobalErrorListeners,
} from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
```

src/app/app.config.ts Parte 2 de 2

```
registerLocaleData(localeEsPe);
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(),
    { provide: LOCALE_ID, useValue: 'es-PE' },
  ],
};
```

Ruta nueva y ruta de edición

/registro abre un formulario vacío. /registro/:id abre el mismo componente y solicita el detalle de ese estudiante. Las rutas cargan los componentes de forma diferida. La ruta comodín debe quedar al final para no interceptar las rutas válidas.

El título de cada ruta describe la vista. El parámetro id llega como cadena a ActivatedRoute. Las direcciones de navegación no contienen el puerto de la API.

src/app/app.routes.ts Parte 1 de 2

```
import type { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', redirectTo: 'resumen', pathMatch: 'full' },
  {
    path: 'resumen',
    title: 'Resumen académico',
    loadComponent: () => import('./paginas/resumen').then((m) => m.Resumen),
  },
  {
    path: 'estudiantes',
    title: 'Estudiantes',
    loadComponent: () =>
      import('./paginas/estudiantes').then((m) => m.Estudiantes),
  },
  {
```

src/app/app.routes.ts Parte 2 de 2

```
    path: 'registro',
    title: 'Registrar estudiante',
    loadComponent: () => import('./paginas/registro').then((m) => m.Registro),
  },
  {
    path: 'registro/:id',
    title: 'Editar estudiante',
    loadComponent: () => import('./paginas/registro').then((m) => m.Registro),
  },
  {
    path: '**',
    title: 'Página no encontrada',
    loadComponent: () =>
      import('./paginas/no-encontrada').then((m) => m.NoEncontrada),
  },
];
```

El punto de entrada arranca el componente raíz con appConfig. El catch muestra en consola un fallo de arranque; no sustituye el manejo de errores de cada solicitud HTTP.

src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

El documento declara idioma, viewport y base de navegación. Conserva public/favicon.ico generado por CLI para resolver el icono referenciado.

src/index.html

```
<!doctype html>
<html lang="es-PE">
  <head>
    <meta charset="utf-8" />
    <title>Portal académico</title>
    <base href="/" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="icon" type="image/x-icon" href="favicon.ico" />
  </head>
  <body>
    <app-root></app-root>
  </body>
</html>
```

Esta vista maneja direcciones del portal sin una ruta válida. Un 404 HTTP del estudiante pertenece al contrato de la API y se gestiona en el formulario; son situaciones distintas.

src/app/paginas/no-encontrada.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  standalone: true,
  imports: [RouterLink],
  template: `
    <h1>Página no encontrada</h1>
    <p>La dirección solicitada no corresponde a una vista del portal.</p>
    <a routerLink="/resumen" class="btn btn-primary">Volver al resumen</a>
  `,
})
export class NoEncontrada {}
```

9 Estructura visual y componentes

Estructura y estilos del portal

El componente raíz contiene navegación, área principal y pie. Cada página se inserta en RouterOutlet. El enlace de salto permite llegar al contenido mediante teclado. La navegación indica la página activa con ariaCurrentWhenActive.

El componente raíz importa RouterLink, RouterLinkActive y RouterOutlet. OnPush y signals favorecen actualizaciones explícitas; el componente no necesita realizar solicitudes de datos.

src/app/app.ts

```
import { ChangeDetectionStrategy, Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  standalone: true,
  imports: [RouterLink, RouterLinkActive, RouterOutlet],
  selector: 'app-root',
  styleUrl: './app.css',
  templateUrl: './app.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class App {}
```

Los enlaces Resumen, Estudiantes y Registro mantienen la continuidad con la semana 8. Durante la edición el enlace Registro no se marca activo porque exige coincidencia exacta.

src/app/app.html Parte 1 de 3

```
<a href="#contenido" class="saltar">Saltar al contenido</a>
<header class="cabecera">
  <div class="container py-4">
    <p class="h4 mb-3">Portal académico</p>
    <nav aria-label="Navegación principal" class="d-flex flex-wrap gap-2">
      <a
        routerLink="/resumen"
        routerLinkActive="activo"
        [routerLinkActiveOptions]="{ exact: true }"
        ariaCurrentWhenActive="page"
        >Resumen</a
      >
```

src/app/app.html Parte 2 de 3

```
      <a
        routerLink="/estudiantes"
        routerLinkActive="activo"
        [routerLinkActiveOptions]="{ exact: true }"
        ariaCurrentWhenActive="page"
        >Estudiantes</a
      >
      <a
        routerLink="/registro"
        routerLinkActive="activo"
        [routerLinkActiveOptions]="{ exact: true }"
        ariaCurrentWhenActive="page"
```

src/app/app.html Parte 3 de 3

```
        >Registro</a
      >
    </nav>
  </div>
</header>
<main id="contenido" tabindex="-1" class="container py-4">
  <router-outlet />
</main>
<footer class="container pb-4 text-body-secondary">
  Laboratorio de Angular conectado a una API REST simulada con datos ficticios.
</footer>
```

Estos estilos pertenecen a la estructura raíz. El enlace de salto queda visible cuando obtiene foco; no se elimina de la navegación por teclado.

src/app/app.css Parte 1 de 2

```
:host {
  display: block;
}
.cabecera {
  background: #172e4d;
  color: #fff;
}
nav a {
  color: #fff;
  padding: 0.5rem 0.8rem;
  border-radius: 0.4rem;
}
nav a.activo {
  background: #fff;
```

src/app/app.css Parte 2 de 2

```
  color: #172e4d;
  font-weight: 700;
}
.saltar {
  position: absolute;
  top: -5rem;
  left: 1rem;
  padding: 0.8rem;
  background: #fff;
  z-index: 10;
}
.saltar:focus {
  top: 0.5rem;
}
```

Bootstrap se importa desde la dependencia local. Los estilos globales definen paneles, formulario y un foco visible, sin incluir scripts de Bootstrap que esta práctica no utiliza.

src/styles.css Parte 1 de 2

```
@import 'bootstrap/dist/css/bootstrap.min.css';

body {
  background: #f4f6f9;
  color: #172e4d;
}
.panel {
  background: #fff;
  padding: 1.5rem;
  border: 1px solid #d7dee8;
  border-radius: 0.8rem;
}
.formulario {
```

src/styles.css Parte 2 de 2

```
  max-width: 42rem;
}
.card {
  border-radius: 0.8rem;
}
a:focus-visible,
button:focus-visible,
input:focus-visible,
select:focus-visible {
  outline: 3px solid #8b4e00;
  outline-offset: 3px;
}
```

Tarjeta y pipe de presentación

La tarjeta recibe un estudiante y el estado de selección mediante inputs. Un output comunica la selección al padre. No conoce la API ni decide si el registro debe eliminarse: esas responsabilidades pertenecen a la página del listado.

input.required<Estudiante>() exige que el padre proporcione el dato. output<Estudiante>() tipa el evento que emitirá el botón.

src/app/componentes/tarjeta-estudiante.ts Parte 1 de 2

```
import { DecimalPipe, PercentPipe } from '@angular/common';
import {
  ChangeDetectionStrategy,
  Component,
  input,
  output,
} from '@angular/core';
import type { Estudiante } from '../modelos/estudiante';
import { EstadoAcademicoPipe } from '../pipes/estado-academico.pipe';

@Component({
```

src/app/componentes/tarjeta-estudiante.ts Parte 2 de 2

```
  selector: 'app-tarjeta-estudiante',
  standalone: true,
  imports: [DecimalPipe, PercentPipe, EstadoAcademicoPipe],
  templateUrl: './tarjeta-estudiante.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class TarjetaEstudiante {
  readonly estudiante = input.required<Estudiante>();
  readonly seleccionado = input(false);
  readonly seleccionar = output<Estudiante>();
}
```

DecimalPipe formatea el promedio. PercentPipe espera una proporción: 94 se divide entre 100 para presentar 94 %. El estado se obtiene mediante el pipe propio; los datos originales no cambian.

src/app/componentes/tarjeta-estudiante.html Parte 1 de 2

```
<article class="card h-100" [class.border-primary]="seleccionado()">
  <div class="card-body">
    <h2 class="h5">{{ estudiante().nombre }}</h2>
    <p class="text-body-secondary">{{ estudiante().programa }}</p>
    <dl>
      <dt>Promedio</dt>
      <dd>{{ estudiante().promedio | number: '1.2-2' }}</dd>
      <dt>Asistencia</dt>
      <dd>{{ estudiante().asistencia / 100 | percent: '1.0-0' }}</dd>
      <dt>Estado académico</dt>
      <dd>{{ estudiante().promedio | estadoAcademico }}</dd>
    </dl>
```

src/app/componentes/tarjeta-estudiante.html Parte 2 de 2

```
    <button
      type="button"
      class="btn btn-outline-primary"
      [attr.aria-pressed]="seleccionado()"
      [attr.aria-label]="'Seleccionar a ' + estudiante().nombre"
      (click)="seleccionar.emit(estudiante())"
    >
      {{ seleccionado() ? 'Seleccionado' : 'Seleccionar' }}
    </button>
  </div>
</article>
```

El pipe transforma un promedio en un texto. Es puro por defecto y llama a la función del modelo. No efectúa solicitudes HTTP ni guarda datos.

src/app/pipes/estado-academico.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';
import { estadoAcademico, EstadoAcademico } from '../modelos/estudiante';

@Pipe({ name: 'estadoAcademico', standalone: true })
export class EstadoAcademicoPipe implements PipeTransform {
  transform(promedio: number): EstadoAcademico {
    return estadoAcademico(promedio);
  }
}
```

10 Consulta y eliminación de estudiantes

Lógica del listado

La página consulta la API al inicializarse. lista guarda la última respuesta correcta; cargando gobierna el estado de lectura; errorCarga muestra fallos de GET. Para eliminar se usan pendiente, eliminando y errorAccion, de modo que un fallo de DELETE no se confunda con uno de lectura.

El botón Eliminar solo marca el registro pendiente. Confirmar llama a DELETE; Cancelar cierra la confirmación sin realizar una solicitud. En el callback next de DELETE se crea un nuevo arreglo con filter. La lista permanece intacta mientras la solicitud está pendiente o si falla.

takeUntilDestroyed cierra las suscripciones al salir de la vista y finalize devuelve el estado de actividad a false. Los métodos bloquean acciones repetidas mientras una solicitud incompatible está en curso.

src/app/paginas/estudiantes.ts Parte 1 de 5

```
import {
  ChangeDetectionStrategy,
  Component,
  DestroyRef,
  inject,
  OnInit,
  signal,
} from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { RouterLink } from '@angular/router';
import { finalize } from 'rxjs';
import { TarjetaEstudiante } from '../componentes/tarjeta-estudiante';
import type { Estudiante } from '../modelos/estudiante';
import { EstudiantesApi } from '../servicios/estudiantes-api';
import { mensajeHttp } from '../servicios/mensaje-http';

@Component({
```

src/app/paginas/estudiantes.ts Parte 2 de 5

```
  standalone: true,
  imports: [RouterLink, TarjetaEstudiante],
  templateUrl: './estudiantes.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class Estudiantes implements OnInit {
  private readonly api = inject(EstudiantesApi);
  private readonly destroyRef = inject(DestroyRef);
  readonly lista = signal<Estudiante[]>([]);
  readonly seleccionado = signal<Estudiante | null>(null);
  readonly pendiente = signal<Estudiante | null>(null);
  readonly cargando = signal(false);
  readonly eliminando = signal(false);
  readonly errorCarga = signal('');
  readonly errorAccion = signal('');
  readonly mensaje = signal('');
  ngOnInit(): void {
```

src/app/paginas/estudiantes.ts Parte 3 de 5

```
    this.cargar();
  }

  cargar(): void {
    if (this.cargando() || this.eliminando()) return;
    this.cargando.set(true);
    this.errorCarga.set('');
    this.api
      .listar()
      .pipe(
        takeUntilDestroyed(this.destroyRef),
        finalize(() => this.cargando.set(false)),
      )
      .subscribe({
        next: (datos) => {
          this.lista.set(datos);
          this.seleccionado.set(null);
```

src/app/paginas/estudiantes.ts Parte 4 de 5

```
        },
        error: (error) => this.errorCarga.set(mensajeHttp(error)),
      });
  }
  seleccionar(estudiante: Estudiante): void {
    this.seleccionado.set(estudiante);
  }
  confirmarEliminacion(): void {
    const e = this.pendiente();
    if (!e || this.eliminando() || this.cargando()) return;
    this.eliminando.set(true);
    this.errorAccion.set('');
    this.mensaje.set('');
    this.api
      .eliminar(e.id)
      .pipe(
        takeUntilDestroyed(this.destroyRef),
```

src/app/paginas/estudiantes.ts Parte 5 de 5

```
      finalize(() => this.eliminando.set(false)),
    )
    .subscribe({
      next: () => {
        this.lista.update((lista) => lista.filter((x) => x.id !== e.id));
        if (this.seleccionado()?.id === e.id) this.seleccionado.set(null);
        this.pendiente.set(null);
        this.mensaje.set(`Se eliminó a ${e.nombre}.`);
      },
      error: (error) => this.errorAccion.set(mensajeHttp(error)),
    });
  }
}
```

Plantilla y estados visibles

La plantilla separa carga y error antes de mostrar los datos. @for sigue cada tarjeta por e.id; @empty informa un resultado vacío correcto. Los enlaces de edición incorporan el id y los botones tienen nombres accesibles que incluyen el estudiante afectado.

La confirmación es una sección visible con botones, no un alert de JavaScript. Los mensajes de resultado usan role=status y los errores role=alert.

src/app/paginas/estudiantes.html Parte 1 de 5

```
<h1>Estudiantes</h1>
<div class="d-flex flex-wrap gap-2 mb-3">
  <a routerLink="/registro" class="btn btn-primary">Registrar estudiante</a>
  <button
    type="button"
    class="btn btn-outline-primary"
    (click)="cargar()"
    [disabled]="cargando() || eliminando()"
  >
    Actualizar listado
  </button>
</div>
<p role="status" aria-live="polite">{{ mensaje() }}</p>
@if (errorAccion()) {
  <p role="alert" class="text-danger">{{ errorAccion() }}</p>
}
@if (cargando()) {
```

src/app/paginas/estudiantes.html Parte 2 de 5

```
  <p role="status">Cargando estudiantes...</p>
} @else if (errorCarga()) {
  <p role="alert" class="text-danger">{{ errorCarga() }}</p>
  <button type="button" class="btn btn-primary" (click)="cargar()">
    Reintentar
  </button>
} @else {
  <p role="status">
    @if (seleccionado(); as e) {
      Seleccionaste a {{ e.nombre }}.
    }
  </p>
  @if (pendiente(); as e) {
    <section class="panel mb-3" aria-labelledby="confirmacion">
      <h2 id="confirmacion" class="h5">Confirmar eliminación</h2>
      <p>Se eliminará a {{ e.nombre }} de la API simulada.</p>
      <button
```

src/app/paginas/estudiantes.html Parte 3 de 5

```
      type="button"
      class="btn btn-danger me-2"
      (click)="confirmarEliminacion()"
      [disabled]="eliminando()"
    >
      {{ eliminando() ? 'Eliminando...' : 'Confirmar' }}
    </button>
    <button
      type="button"
      class="btn btn-outline-secondary"
      (click)="pendiente.set(null)"
      [disabled]="eliminando()"
    >
      Cancelar
    </button>
  </section>
}
```

src/app/paginas/estudiantes.html Parte 4 de 5

```
<div class="row g-3">
  @for (e of lista(); track e.id) {
    <div class="col-md-6 col-xl-4">
      <app-tarjeta-estudiante
        [estudiante]="e"
        [seleccionado]="seleccionado()?.id === e.id"
        (seleccionar)="seleccionar($event)"
      />
      <div class="d-flex gap-2 my-2">
        <a
          [routerLink]="['/registro', e.id]"
          class="btn btn-outline-primary"
          [attr.aria-label]="'Editar a ' + e.nombre"
          >Editar</a
        >
        <button
          type="button"
```

src/app/paginas/estudiantes.html Parte 5 de 5

```
          class="btn btn-outline-danger"
          (click)="pendiente.set(e)"
          [disabled]="eliminando()"
          [attr.aria-label]="'Eliminar a ' + e.nombre"
        >
          Eliminar
        </button>
      </div>
    </div>
  } @empty {
    <p>No hay estudiantes registrados. Agrega el primero.</p>
  }
</div>
}
```

Comprueba el avance. La vista muestra seis tarjetas al arrancar con la semilla. Pulsar Eliminar y luego Cancelar no debe generar un DELETE en Network.

11 Registro y edición de estudiantes

Validación y datos enviados

El formulario mantiene cuatro controles tipados. Los controles numéricos admiten null para representar un campo sin completar. El valor cero es válido; no uses `if (!promedio)` para comprobar ausencia. Los validadores required, min, max y numeroFinito se complementan para impedir valores ausentes, fuera de rango o no finitos. [R9]

El nombre se evalúa después de quitar los espacios exteriores y se recorta también en el cuerpo enviado. Un formulario inválido marca todos los controles como tocados y termina antes de construir una solicitud. `novalidate` permite que Angular gestione los mensajes; no elimina sus validadores.

Consultar editar y guardar

ActivatedRoute.paramMap comunica el id de la ruta. La página reinicia y deshabilita el formulario antes de consultar. Si hay id realiza GET; si no lo hay, emite null con of y habilita un registro nuevo. Si el GET de edición devuelve 404, muestra el error y mantiene el formulario deshabilitado para evitar crear o modificar un registro incorrecto.

Al guardar, se selecciona POST para un registro nuevo o PATCH para uno existente. `getRawValue()` obtiene todos los valores tipados; el id se toma de la ruta y no se incluye en el cuerpo. El formulario queda deshabilitado mientras se guarda y finalize lo habilita al terminar. El éxito de POST limpia los controles; un error conserva lo escrito.

Lee el archivo por responsabilidades: validadores, estado, formulario, carga por parámetro y guardado. El catchError de la consulta de edición devuelve null únicamente después de registrar el error; no se interpreta como un estudiante válido.

src/app/paginas/registro.ts Parte 1 de 9

```
import {
  ChangeDetectionStrategy,
  Component,
  DestroyRef,
  inject,
  OnInit,
  signal,
} from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import {
  AbstractControl,
  FormBuilder,
  ReactiveFormsModule,
  Validators,
} from '@angular/forms';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { catchError, finalize, of, switchMap, tap } from 'rxjs';
```

src/app/paginas/registro.ts Parte 2 de 9

```
import type { Programa } from '../modelos/estudiante';
import { EstudiantesApi } from '../servicios/estudiantes-api';
import { mensajeHttp } from '../servicios/mensaje-http';

function nombreValido(control: AbstractControl) {
  const texto = String(control.value ?? '').trim();
  return texto.length >= 3 && texto.length <= 80 ? null : { nombre: true };
}
function numeroFinito(control: AbstractControl) {
  return control.value === null || Number.isFinite(control.value)
    ? null
    : { finito: true };
}

@Component({
  standalone: true,
  imports: [ReactiveFormsModule, RouterLink],
```

src/app/paginas/registro.ts Parte 3 de 9

```
  templateUrl: './registro.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class Registro implements OnInit {
  private readonly fb = inject(FormBuilder);
  private readonly api = inject(EstudiantesApi);
  private readonly route = inject(ActivatedRoute);
  private readonly destroyRef = inject(DestroyRef);
  readonly id = signal<string | null>(null);
  readonly cargando = signal(false);
  readonly guardando = signal(false);
  readonly enviado = signal(false);
  readonly mensaje = signal('');
  readonly errorCarga = signal('');
  readonly errorGuardado = signal('');
  readonly form = this.fb.group({
    nombre: this.fb.nonNullable.control('', [nombreValido]),
```

src/app/paginas/registro.ts Parte 4 de 9

```
    programa: this.fb.nonNullable.control<Programa>('Software', [
      Validators.required,
    ]),
    promedio: this.fb.control<number | null>(null, [
      Validators.required,
      Validators.min(0),
      Validators.max(20),
      numeroFinito,
    ]),
    asistencia: this.fb.control<number | null>(null, [
      Validators.required,
      Validators.min(0),
      Validators.max(100),
      numeroFinito,
    ]),
  });
```

src/app/paginas/registro.ts Parte 5 de 9

```
ngOnInit(): void {
  this.route.paramMap
    .pipe(
      tap((params) => {
        this.id.set(params.get('id'));
        this.cargando.set(true);
        this.errorCarga.set('');
        this.errorGuardado.set('');
        this.mensaje.set('');
        this.enviado.set(false);
        this.form.reset();
        this.form.disable();
      }),
      switchMap(() => {
        const id = this.id();
        return id
          ? this.api.obtener(id).pipe(
```

src/app/paginas/registro.ts Parte 6 de 9

```
              catchError((error) => {
                this.errorCarga.set(mensajeHttp(error));
                return of(null);
              }),
            )
          : of(null);
      }),
      takeUntilDestroyed(this.destroyRef),
    )
    .subscribe((estudiante) => {
      if (estudiante) this.form.patchValue(estudiante);
      if (!this.errorCarga()) this.form.enable();
      this.cargando.set(false);
    });
}

invalido(campo: keyof typeof this.form.controls): boolean {
```

src/app/paginas/registro.ts Parte 7 de 9

```
  const control = this.form.controls[campo];
  return control.invalid && (control.touched || this.enviado());
}

guardar(): void {
  if (this.guardando() || this.cargando() || this.errorCarga()) return;
  this.enviado.set(true);
  this.mensaje.set('');
  this.errorGuardado.set('');
  if (this.form.invalid) {
    this.form.markAllAsTouched();
    return;
  }
  const valores = this.form.getRawValue();
  if (valores.promedio === null || valores.asistencia === null) return;
  const datos = {
    ...valores,
```

src/app/paginas/registro.ts Parte 8 de 9

```
    nombre: valores.nombre.trim(),
    promedio: valores.promedio,
    asistencia: valores.asistencia,
  };
  const id = this.id();
  const solicitud = id
    ? this.api.actualizar(id, datos)
    : this.api.crear(datos);
  this.guardando.set(true);
  this.form.disable();
  solicitud
    .pipe(
      takeUntilDestroyed(this.destroyRef),
      finalize(() => {
        this.guardando.set(false);
        this.form.enable();
      }),
```

src/app/paginas/registro.ts Parte 9 de 9

```
    )
    .subscribe({
      next: (estudiante) => {
        this.mensaje.set(
          `Se guardó a ${estudiante.nombre} con id ${estudiante.id}.`,
        );
        if (!id) this.form.reset();
        this.enviado.set(false);
      },
      error: (error) => this.errorGuardado.set(mensajeHttp(error)),
    });
  }
}
```

Plantilla accesible

Cada control tiene label, id y un mensaje asociado con aria-describedby. aria-invalid comunica cuándo está inválido. Los mensajes se muestran después de tocar un campo o intentar enviar el formulario. El botón no se bloquea por formulario inválido: al pulsarlo el estudiante recibe la explicación de qué corregir.

La plantilla indica si estás registrando o editando. El texto Guardando aparece mientras la solicitud está pendiente. El mensaje de éxito muestra el identificador real que devolvió el servidor.

src/app/paginas/registro.html Parte 1 de 7

```
<h1>{{ id() ? 'Editar estudiante' : 'Registrar estudiante' }}</h1>
<p>Los cambios se guardan en la API simulada.</p>
@if (cargando()) {
  <p role="status">Cargando registro...</p>
}
@if (errorCarga()) {
  <p role="alert" class="text-danger">{{ errorCarga() }}</p>
}
<form
  [formGroup]="form"
  (ngSubmit)="guardar()"
  novalidate
  class="panel formulario"
>
  <div class="mb-3">
    <label for="nombre" class="form-label">Nombre completo</label>
    <input
```

src/app/paginas/registro.html Parte 2 de 7

```
      id="nombre"
      formControlName="nombre"
      type="text"
      autocomplete="name"
      class="form-control"
      [class.is-invalid]="invalido('nombre')"
      [attr.aria-invalid]="invalido('nombre')"
      aria-describedby="nombre-ayuda nombre-error"
    />
    <p id="nombre-ayuda" class="form-text">
      Entre 3 y 80 caracteres después de quitar espacios exteriores.
    </p>
    <p id="nombre-error" class="text-danger">
      @if (invalido('nombre')) {
        Escribe un nombre de 3 a 80 caracteres.
      }
    </p>
```

src/app/paginas/registro.html Parte 3 de 7

```
  </div>
  <div class="mb-3">
    <label for="programa" class="form-label">Programa</label>
    <select
      id="programa"
      formControlName="programa"
      class="form-select"
      [attr.aria-invalid]="invalido('programa')"
      aria-describedby="programa-error"
    >
      <option value="Software">Ingeniería de Software</option>
      <option value="Sistemas">Ingeniería de Sistemas</option>
    </select>
    <p id="programa-error" class="text-danger">
      @if (invalido('programa')) {
        Selecciona un programa.
      }
```

src/app/paginas/registro.html Parte 4 de 7

```
  </p>
</div>
<div class="mb-3">
  <label for="promedio" class="form-label">Promedio de 0 a 20</label>
  <input
    id="promedio"
    formControlName="promedio"
    type="number"
    min="0"
    max="20"
    step="any"
    class="form-control"
    [class.is-invalid]="invalido('promedio')"
    [attr.aria-invalid]="invalido('promedio')"
    aria-describedby="promedio-error"
  />
  <p id="promedio-error" class="text-danger">
```

src/app/paginas/registro.html Parte 5 de 7

```
    @if (invalido('promedio')) {
      Ingresa un número entre 0 y 20.
    }
  </p>
</div>
<div class="mb-3">
  <label for="asistencia" class="form-label"
    >Asistencia de 0 a 100 por ciento</label
  >
  <input
    id="asistencia"
    formControlName="asistencia"
    type="number"
    min="0"
    max="100"
    step="any"
    class="form-control"
```

src/app/paginas/registro.html Parte 6 de 7

```
    [class.is-invalid]="invalido('asistencia')"
    [attr.aria-invalid]="invalido('asistencia')"
    aria-describedby="asistencia-error"
  />
  <p id="asistencia-error" class="text-danger">
    @if (invalido('asistencia')) {
      Ingresa un número entre 0 y 100.
    }
  </p>
</div>
@if (enviado() && form.invalid) {
  <p role="alert" class="text-danger">
    Revisa los campos señalados antes de guardar.
  </p>
}
<div class="d-flex flex-wrap gap-2">
  <button
```

src/app/paginas/registro.html Parte 7 de 7

```
      type="submit"
      class="btn btn-primary"
      [disabled]="cargando() || guardando() || !!errorCarga()"
    >
      {{ guardando() ? 'Guardando...' : 'Guardar estudiante' }}
    </button>
    <a routerLink="/estudiantes" class="btn btn-outline-secondary"
      >Ver estudiantes</a
    >
  </div>
</form>
<p role="status" aria-live="polite" class="mt-3">{{ mensaje() }}</p>
@if (errorGuardado()) {
  <p role="alert" class="text-danger">{{ errorGuardado() }}</p>
}
```

Comprueba el avance. Enviar los campos vacíos debe mostrar errores y no producir POST. Crea Elena Ruiz con Software, promedio 16 y asistencia 90; el id será el de la respuesta, no uno calculado por el navegador.

Si sales de la página durante una modificación, el cliente puede cancelar su suscripción, pero el servidor puede haber recibido ya la operación. Vuelve al listado y actualízalo antes de asumir que el cambio no ocurrió. El laboratorio bloquea duplicados simultáneos en el formulario; no implementa claves de idempotencia ni coordinación entre varios usuarios.

12 Resumen académico desde la API

Cálculos derivados de la respuesta

Resumen vuelve a consultar el listado al entrar en la ruta. El total y el promedio general se derivan con computed a partir de la lista recibida. No es necesario mantener otro promedio mutable. Si la lista está vacía no se divide entre cero y la plantilla informa que no hay estudiantes.

La página distingue carga y error y permite reintentar. La fecha identifica la consulta mostrada por la vista; no es una fecha de registro de cada estudiante ni un dato enviado por la API.

src/app/paginas/resumen.ts Parte 1 de 4

```
import { DatePipe, DecimalPipe } from '@angular/common';
import {
  ChangeDetectionStrategy,
  Component,
  computed,
  DestroyRef,
  inject,
  OnInit,
  signal,
} from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { RouterLink } from '@angular/router';
import { finalize } from 'rxjs';
import type { Estudiante } from '../modelos/estudiante';
```

src/app/paginas/resumen.ts Parte 2 de 4

```
import { EstudiantesApi } from '../servicios/estudiantes-api';
import { mensajeHttp } from '../servicios/mensaje-http';

@Component({
  standalone: true,
  imports: [DatePipe, DecimalPipe, RouterLink],
  templateUrl: './resumen.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class Resumen implements OnInit {
  private readonly api = inject(EstudiantesApi);
  private readonly destroyRef = inject(DestroyRef);
  readonly lista = signal<Estudiante[]>([]);
  readonly cargando = signal(false);
```

src/app/paginas/resumen.ts Parte 3 de 4

```
  readonly error = signal('');
  readonly fecha = new Date();
  readonly total = computed(() => this.lista().length);
  readonly promedio = computed(() =>
    this.total()
      ? this.lista().reduce((suma, e) => suma + e.promedio, 0) / this.total()
      : 0,
  );
  ngOnInit(): void {
    this.cargar();
  }

  cargar(): void {
    if (this.cargando()) return;
```

src/app/paginas/resumen.ts Parte 4 de 4

```
    this.cargando.set(true);
    this.error.set('');
    this.api
      .listar()
      .pipe(
        takeUntilDestroyed(this.destroyRef),
        finalize(() => this.cargando.set(false)),
      )
      .subscribe({
        next: (datos) => this.lista.set(datos),
        error: (error) => this.error.set(mensajeHttp(error)),
      });
  }
}
```

El promedio se presenta con dos decimales usando el locale es-PE. Al navegar al resumen después de un cambio se realiza un nuevo GET; la práctica no mantiene una caché compartida entre páginas.

src/app/paginas/resumen.html Parte 1 de 2

```
<h1>Resumen académico</h1>
<p>Consulta del {{ fecha | date: 'dd/MM/yyyy' }} con datos de la API.</p>
@if (cargando()) {
  <p role="status">Cargando resumen...</p>
} @else if (error()) {
  <p role="alert" class="text-danger">{{ error() }}</p>
  <button type="button" class="btn btn-primary" (click)="cargar()">
    Reintentar
  </button>
} @else if (total() === 0) {
  <p>No hay estudiantes registrados.</p>
} @else {
  <div class="row g-3 my-3">
    <div class="col-md-6">
      <section class="panel" aria-labelledby="total-titulo">
```

src/app/paginas/resumen.html Parte 2 de 2

```
        <h2 id="total-titulo" class="h5">Estudiantes registrados</h2>
        <p class="display-6">{{ total() }}</p>
      </section>
    </div>
    <div class="col-md-6">
      <section class="panel" aria-labelledby="promedio-titulo">
        <h2 id="promedio-titulo" class="h5">Promedio general</h2>
        <p class="display-6">{{ promedio() | number: '1.2-2' }}</p>
      </section>
    </div>
  </div>
}
<a routerLink="/estudiantes" class="btn btn-outline-primary mt-3"
  >Consultar estudiantes</a
>
```

Comprueba el avance. Con los seis registros iniciales el resumen muestra total 6 y promedio 13,32. La suma de sus promedios es 79,9; 79,9 dividido entre 6 se redondea solo para presentarlo.

Iniciar Angular y revisar el recorrido

1. Mantén abierta la terminal A con la API. En una terminal B, en la misma raíz portal-rest, ejecuta npm start.
2. Abre la URL que muestre Angular CLI, normalmente http://localhost:4200. La ruta inicial debe redirigir a Resumen.
3. Abre Estudiantes y confirma que recibes los seis registros. Abre Network y filtra por Fetch/XHR para ver GET /estudiantes.
4. Abre Registro y prueba el envío vacío; luego realiza un registro válido. Entra en Estudiantes para consultar la lista desde la API.
5. Edita el registro creado y confirma un PATCH; solicita Eliminar, prueba Cancelar y finalmente confirma DELETE.

Comando en la raíz del proyecto

```
npm start
```

13 Pruebas automatizadas del portal

Qué comprueban las pruebas

Las pruebas HTTP sustituyen el backend de HttpClient por uno controlado. No necesitan tener JSON Server en ejecución. expectOne() exige una sola solicitud coincidente; flush() entrega una respuesta simulada y verify() verifica que no queden solicitudes inesperadas. Si declaras ambos proveedores, provideHttpClientTesting debe ir después de provideHttpClient. [R10]

El proyecto usa Vitest mediante el builder de Angular CLI. jsdom proporciona un DOM simulado para crear componentes y revisar su contenido. Estas comprobaciones no sustituyen la revisión de teclado, apariencia y comportamiento en un navegador real. [R11]

Suite	Cantidad	Qué verifica
App	2	Creación de la estructura, navegación y contenedor de rutas.
EstudiantesApi	8	GET frío, id codificado, POST, PATCH, PUT, DELETE, error 404 y cancelación.
Vistas con HTTP	9	Carga, vacío, error, reintento, eliminación, validación, duplicados y edición.

Prueba de la estructura principal

Reemplaza la prueba de la plantilla inicial que generó CLI. Esta suite comprueba que el portal expone sus tres enlaces y conserva RouterOutlet.

src/app/app.spec.ts Parte 1 de 2

```
import { TestBed } from '@angular/core/testing';
import { App } from './app';
import { appConfig } from './app.config';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
      providers: [...appConfig.providers],
    }).compileComponents();
  });

  it('crea la estructura principal', () => {
```

src/app/app.spec.ts Parte 2 de 2

```
    const fixture = TestBed.createComponent(App);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });
  it('ofrece tres enlaces y un contenedor de rutas', async () => {
    const fixture = TestBed.createComponent(App);
    await fixture.whenStable();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelectorAll('nav a')).toHaveLength(3);
    expect(compiled.querySelector('router-outlet')).not.toBeNull();
  });
});
```

Pruebas del servicio HTTP

La primera prueba construye el Observable sin suscribirse y exige que todavía no haya solicitud. Después se suscribe, inspecciona el método y entrega un arreglo. Las pruebas de modificación inspeccionan los cuerpos y confirman que el cliente no fabrica el id.

El caso de DELETE entrega un cuerpo nulo para comprobar que el servicio no depende de sus campos. El caso de cancelación verifica req.cancelled tras cerrar la suscripción, sin afirmar que un servidor desharía una operación.

src/app/servicios/estudiantes-api.spec.ts Parte 1 de 6

```
import { provideHttpClient } from '@angular/common/http';
import {
  HttpTestingController,
  provideHttpClientTesting,
} from '@angular/common/http/testing';
import { TestBed } from '@angular/core/testing';
import { API_URL, EstudiantesApi } from './estudiantes-api';
import { mensajeHttp } from './mensaje-http';
import type { BorradorEstudiante, Estudiante } from '../modelos/estudiante';

const datos: BorradorEstudiante = {
  nombre: 'Elena Ruiz',
  programa: 'Software',
  promedio: 16,
  asistencia: 90,
};
```

src/app/servicios/estudiantes-api.spec.ts Parte 2 de 6

```
const estudiante: Estudiante = { ...datos, id: 'nuevo-id' };

describe('EstudiantesApi', () => {
  let api: EstudiantesApi;
  let http: HttpTestingController;
  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient(), provideHttpClientTesting()],
    });
    api = TestBed.inject(EstudiantesApi);
    http = TestBed.inject(HttpTestingController);
  });
  afterEach(() => http.verify());

  it('envía GET solo al suscribirse', () => {
    const solicitud = api.listar();
```

src/app/servicios/estudiantes-api.spec.ts Parte 3 de 6

```
  http.expectNone(API_URL);
  solicitud.subscribe((lista) => expect(lista).toEqual([estudiante]));
  const req = http.expectOne(API_URL);
  expect(req.request.method).toBe('GET');
  req.flush([estudiante]);
 });
 it('consulta un identificador y codifica su segmento', () => {
  api.obtener('a b').subscribe();
  const req = http.expectOne(`${API_URL}/a%20b`);
  expect(req.request.method).toBe('GET');
  req.flush(estudiante);
 });
 it('crea con POST sin fabricar el id', () => {
  api.crear(datos).subscribe((e) => expect(e.id).toBe('nuevo-id'));
  const req = http.expectOne(API_URL);
  expect(req.request.method).toBe('POST');
```

src/app/servicios/estudiantes-api.spec.ts Parte 4 de 6

```
  expect(req.request.body).toEqual(datos);
  expect(req.request.body.id).toBeUndefined();
  req.flush(estudiante, { status: 201, statusText: 'Created' });
 });
 it('envía un cambio parcial con PATCH', () => {
  api.actualizar('101', { asistencia: 95 }).subscribe();
  const req = http.expectOne(`${API_URL}/101`);
  expect(req.request.method).toBe('PATCH');
  expect(req.request.body).toEqual({ asistencia: 95 });
  req.flush({ ...estudiante, asistencia: 95 });
 });
 it('envía la representación completa con PUT', () => {
  api.reemplazar('101', datos).subscribe();
  const req = http.expectOne(`${API_URL}/101`);
  expect(req.request.method).toBe('PUT');
  expect(req.request.body).toEqual(datos);
```

src/app/servicios/estudiantes-api.spec.ts Parte 5 de 6

```
  req.flush({ ...datos, id: '101' });
 });
 it('elimina sin depender del cuerpo de respuesta', () => {
  api.eliminar('101').subscribe();
  const req = http.expectOne(`${API_URL}/101`);
  expect(req.request.method).toBe('DELETE');
  req.flush(null);
 });
 it('comunica un error 404 por el canal de error', () => {
  api.obtener('inexistente').subscribe({
   next: () => {
    throw new Error('No debe emitir éxito');
   },
   error: (error) => expect(mensajeHttp(error)).toContain('ya no existe'),
  });
  http.expectOne(`${API_URL}/inexistente`).flush(
```

src/app/servicios/estudiantes-api.spec.ts Parte 6 de 6

```
      {},
      {
        status: 404,
        statusText: 'Not Found',
      },
    );
  });
  it('cancela la solicitud al cerrar la suscripción', () => {
    const sub = api.listar().subscribe();
    const req = http.expectOne(API_URL);
    sub.unsubscribe();
    expect(req.cancelled).toBe(true);
  });
});
```

Pruebas de las páginas

El BehaviorSubject simula los cambios de paramMap en la ruta de registro. Las pruebas crean el componente, ejecutan su ciclo de inicialización y entregan respuestas controladas. El caso de cero confirma que una entrada numérica válida no se trata como vacía y que el segundo intento de guardar no produce otro POST.

Se comprueban tanto el estado de TypeScript como un mensaje de la plantilla. Los fallos simulados permiten reproducir un 500 sin añadir rutas artificiales a la API de datos.

src/app/paginas/portal-http.spec.ts Parte 1 de 10

```
import { provideHttpClient } from '@angular/common/http';
import {
  HttpTestingController,
  provideHttpClientTesting,
} from '@angular/common/http/testing';
import { TestBed } from '@angular/core/testing';
import {
  ActivatedRoute,
  convertToParamMap,
  provideRouter,
} from '@angular/router';
import { BehaviorSubject } from 'rxjs';
import { API_URL } from '../servicios/estudiantes-api';
import { Estudiantes } from './estudiantes';
import { Registro } from './registro';
import type { Estudiante } from '../modelos/estudiante';
```

src/app/paginas/portal-http.spec.ts Parte 2 de 10

```
const ana: Estudiante = {
  id: '101',
  nombre: 'Ana Torres',
  programa: 'Software',
  promedio: 17.2,
  asistencia: 94,
};
describe('Vistas con HTTP', () => {
  let http: HttpTestingController;
  let params: BehaviorSubject<ReturnType<typeof convertToParamMap>>;
  beforeEach(() => {
    params = new BehaviorSubject(convertToParamMap({}));
    TestBed.configureTestingModule({
      providers: [
        provideRouter([]),
```

src/app/paginas/portal-http.spec.ts Parte 3 de 10

```
      provideHttpClient(),
      provideHttpClientTesting(),
      {
        provide: ActivatedRoute,
        useValue: { paramMap: params.asObservable() },
      },
    ],
  });
  http = TestBed.inject(HttpTestingController);
});
afterEach(() => http.verify());

it('muestra carga y acepta un listado vacío', () => {
  const f = TestBed.createComponent(Estudiantes);
  f.detectChanges();
  expect(f.componentInstance.cargando()).toBe(true);
```

src/app/paginas/portal-http.spec.ts Parte 4 de 10

```
  http.expectOne(API_URL).flush([]);
  f.detectChanges();
  expect(f.componentInstance.cargando()).toBe(false);
  expect(f.nativeElement.textContent).toContain('No hay estudiantes');
});
it('muestra el fallo de red y permite reintentar', () => {
  const f = TestBed.createComponent(Estudiantes);
  f.detectChanges();
  http.expectOne(API_URL).error(new ProgressEvent('error'));
  expect(f.componentInstance.errorCarga()).toContain('No se pudo contactar');
  expect(f.componentInstance.cargando()).toBe(false);
  f.componentInstance.cargar();
  http.expectOne(API_URL).flush([ana]);
  expect(f.componentInstance.lista()).toEqual([ana]);
});
it('retira un registro solo después de confirmar el DELETE', () => {
```

src/app/paginas/portal-http.spec.ts Parte 5 de 10

```
  const f = TestBed.createComponent(Estudiantes);
  f.detectChanges();
  http.expectOne(API_URL).flush([ana]);
  f.componentInstance.pendiente.set(ana);
  f.componentInstance.confirmarEliminacion();
  expect(f.componentInstance.lista()).toHaveLength(1);
  http.expectOne(`${API_URL}/101`).flush(null);
  expect(f.componentInstance.lista()).toHaveLength(0);
  expect(f.componentInstance.eliminando()).toBe(false);
});
it('conserva la tarjeta cuando falla DELETE', () => {
  const f = TestBed.createComponent(Estudiantes);
  f.detectChanges();
  http.expectOne(API_URL).flush([ana]);
  f.componentInstance.pendiente.set(ana);
  f.componentInstance.confirmarEliminacion();
```

src/app/paginas/portal-http.spec.ts Parte 6 de 10

```
    http
      .expectOne(`${API_URL}/101`)
      .flush({}, { status: 500, statusText: 'Error' });
    expect(f.componentInstance.lista()).toHaveLength(1);
    expect(f.componentInstance.errorAccion()).toContain('La API falló');
  });
  it('rechaza un formulario vacío sin emitir POST', () => {
    const f = TestBed.createComponent(Registro);
    f.detectChanges();
    f.componentInstance.guardar();
    expect(f.componentInstance.form.invalid).toBe(true);
    http.expectNone(API_URL);
  });
  it('acepta cero y bloquea envíos duplicados durante POST', () => {
    const f = TestBed.createComponent(Registro);
    f.detectChanges();
```

src/app/paginas/portal-http.spec.ts Parte 7 de 10

```
    f.componentInstance.form.setValue({
      nombre: ' Elena Ruiz ',
      programa: 'Software',
      promedio: 0,
      asistencia: 0,
    });
    f.componentInstance.guardar();
    f.componentInstance.guardar();
    const req = http.expectOne(API_URL);
    expect(req.request.body.nombre).toBe('Elena Ruiz');
    expect(req.request.body.promedio).toBe(0);
    expect(f.componentInstance.guardando()).toBe(true);
    req.flush({ ...req.request.body, id: 'nuevo-id' });
    expect(f.componentInstance.mensaje()).toContain('nuevo-id');
    expect(f.componentInstance.form.controls.promedio.value).toBeNull();
    expect(f.componentInstance.guardando()).toBe(false);
```

src/app/paginas/portal-http.spec.ts Parte 8 de 10

```
  });
  it('conserva los valores y habilita el formulario si falla POST', () => {
    const f = TestBed.createComponent(Registro);
    f.detectChanges();
    f.componentInstance.form.setValue({
      nombre: 'Elena Ruiz',
      programa: 'Software',
      promedio: 16,
      asistencia: 90,
    });
    f.componentInstance.guardar();
    http.expectOne(API_URL).flush({}, { status: 500, statusText: 'Error' });
    expect(f.componentInstance.form.controls.nombre.value).toBe('Elena Ruiz');
    expect(f.componentInstance.form.enabled).toBe(true);
    expect(f.componentInstance.errorGuardado()).toContain('La API falló');
  });
```

src/app/paginas/portal-http.spec.ts Parte 9 de 10

```
  it('carga para editar y envía PATCH sin cambiar el id', () => {
    params.next(convertToParamMap({ id: '101' }));
    const f = TestBed.createComponent(Registro);
    f.detectChanges();
    http.expectOne(`${API_URL}/101`).flush(ana);
    f.componentInstance.form.controls.promedio.setValue(18);
    f.componentInstance.guardar();
    const req = http.expectOne(`${API_URL}/101`);
    expect(req.request.method).toBe('PATCH');
    expect(req.request.body.id).toBeUndefined();
    expect(req.request.body.promedio).toBe(18);
    req.flush({ ...ana, promedio: 18 });
  });
  it('impide guardar un id inexistente', () => {
    params.next(convertToParamMap({ id: 'inexistente' }));
    const f = TestBed.createComponent(Registro);
```

src/app/paginas/portal-http.spec.ts Parte 10 de 10

```
    f.detectChanges();
    http.expectOne(`${API_URL}/inexistente`).flush(
      {},
      {
        status: 404,
        statusText: 'Not Found',
      },
    );
    f.componentInstance.guardar();
    expect(f.componentInstance.form.disabled).toBe(true);
    http.expectNone((req) => req.method === 'POST' || req.method === 'PATCH');
  });
});
```

Ejecutar y leer el resultado

Comando en la raíz del proyecto

```
npm run verify
```

El script compila primero la aplicación para producción y, si termina correctamente, ejecuta las pruebas una vez. Debes obtener 3 archivos de pruebas aprobados y 19 pruebas aprobadas. Los tiempos, hashes y tamaños pueden variar según el equipo; no copies un tiempo de ejecución como criterio de calidad.

La aplicación y los archivos incluidos superaron la compilación estricta y las 19 pruebas. Además, las operaciones se comprobaron contra la API local en ejecución, incluidos PUT, PATCH, DELETE, CORS y conservación del archivo al reiniciar el servidor. Completa también la matriz manual siguiente para comprobar tu propia instalación.

Si una prueba falla, localiza la primera expectativa fallida, compara método, URL y cuerpo, y corrige la implementación o la configuración que incumple el contrato. No borres la expectativa ni la conviertas en una afirmación que siempre pasa.

14 Pruebas manuales y persistencia

Matriz de comprobación del portal

Empieza con la semilla inicial. Para restaurarla, detén la API con Ctrl+C, ejecuta npm run api:reset y vuelve a iniciar npm run api. Mantén Angular en la terminal B. Registra en tus evidencias el estado HTTP y el resultado visible, además de una captura de pantalla.

Caso	Acción	Resultado esperado
M1	Abrir Resumen con la semilla.	6 estudiantes y promedio 13,32.
M2	Abrir Estudiantes y seleccionar Ana.	6 tarjetas y mensaje de selección; seleccionar no envía HTTP.
M3	Enviar Registro sin completar los campos.	Mensajes de validación; ninguna solicitud POST.
M4	Registrar Elena Ruiz, Software, 16 y 90.	Un POST 201 con id de cadena; 7 registros y promedio 13,70 en Resumen.
M5	Recargar y reiniciar solo la API después de M4.	Elena sigue disponible; no se ejecutó api:reset.
M6	Editar Elena y cambiar promedio a 18.	Un PATCH; mismo id; 7 registros y promedio 13,99 en Resumen.
M7	Solicitar Eliminar a Elena y pulsar Cancelar.	Elena permanece y no se envía DELETE.
M8	Confirmar la eliminación de Elena.	DELETE correcto; 6 registros y promedio 13,32.
M9	Visitar /registro/inexistente.	GET 404, mensaje de error y formulario deshabilitado.
M10	Enviar nombre de dos caracteres o promedio 21.	Mensajes en los controles; no se envía POST ni PATCH.
M11	Guardar un registro con promedio 0 y asistencia 0.	Valores aceptados y conservados como números; elimina luego ese registro de prueba.
M12	Navegar por Tab y activar botones con teclado.	Foco visible, labels asociados, confirmación y mensajes comprensibles.

Los valores del resumen se calculan con la lista completa: M4 da (79,9 + 16) / 7 = 13,70; M6 da (79,9 + 18) / 7 = 13,99 al mostrar dos decimales. La API conserva números sin aplicar este redondeo de presentación.

Comprobar persistencia sin confundirla con estado local

6. Crea Elena y copia el id recibido en Network o en el mensaje de éxito.
7. Recarga el navegador. Consulta /estudiantes en la API y comprueba que el objeto está presente.
8. Abre api/db.json en el editor y localiza el id. No lo edites mientras la API está escribiendo.
9. Detén y reinicia únicamente la API, sin ejecutar api:reset. Consulta de nuevo el id y confirma que conserva el registro.
10. Vuelve al listado del portal y pulsa Actualizar listado. Documenta el GET y el registro recibido.

Demostrar la diferencia entre PATCH y PUT

Usa un cliente HTTP disponible para enviar solicitudes a la API local, o apóyate en las pruebas del servicio de la sección 13. Trabaja con un registro ficticio creado para esta actividad. Sustituye ID_REAL en las URL por su id; no envíes ese rótulo literalmente.

Solicitud de cambio parcial

```
PATCH http://127.0.0.1:3000/estudiantes/ID_REAL
Content-Type: application/json

{ "asistencia": 95, "observacion": "Prueba de reemplazo" }
```

Consulta ese id con GET. El nombre, el programa y el promedio deben conservarse y aparecerá observacion. Esa propiedad adicional permite ver el efecto del reemplazo, aunque no forme parte del modelo de Angular. Realiza esta actividad desde la API y elimina al terminar el registro de demostración.

Solicitud de representación completa

```
PUT http://127.0.0.1:3000/estudiantes/ID_REAL
Content-Type: application/json

{
  "nombre": "Elena Ruiz",
  "programa": "Software",
  "promedio": 16,
  "asistencia": 90
}
```

Consulta nuevamente el id. En el simulador fijado, PUT conserva el identificador de la URL, reemplaza los campos editables y elimina observacion al no incluirla. Comprueba el resultado JSON: no basta con mirar el formulario, que no presenta esa propiedad. Al terminar, elimina el registro ficticio de demostración para recuperar el conjunto de datos esperado.

Latencia fallos y recuperación

1. En DevTools abre Network, activa una limitación de velocidad como Slow 3G y actualiza el listado. Observa Cargando estudiantes antes de las tarjetas. Desactiva la limitación al terminar.
2. Con la API detenida, actualiza el listado. Debes ver el error de contacto y Reintentar. Reinicia la API y pulsa Reintentar: deben volver los datos.
3. Completa un registro válido y detén la API antes de pulsar Guardar. Comprueba que los valores permanecen tras el error y que el formulario se habilita nuevamente.
4. Con latencia activa y la API disponible, pulsa Guardar dos veces rápidamente. Debe iniciarse un solo POST mientras la operación está pendiente.
5. Para el estado vacío y los fallos 500 utiliza las pruebas automatizadas incluidas; no necesitas borrar todos los registros ni modificar el servidor para reproducirlos.

No uses el antiguo parámetro --delay de ejemplos de JSON Server 0.x: la línea 1 recomienda limitar la velocidad desde las herramientas del navegador. Si se interrumpe una respuesta de POST después de llegar al servidor, vuelve a consultar la lista antes de repetir el registro. [R3]

15 Depuración y errores frecuentes

Inspección de una solicitud

1. Abre DevTools antes de realizar la operación y entra en Network. Selecciona Fetch/XHR y, si necesitas conservar el recorrido, activa Preserve log.
2. Realiza una sola acción del portal y selecciona su solicitud. Confirma Request URL, Request Method y Status Code.

3. En Payload revisa los campos enviados. POST y PATCH no deben incluir un id fabricado por el formulario.
4. En Response comprueba si recibes un arreglo, un objeto o una respuesta sin datos según la operación.
5. Si falla, revisa la consola y las terminales. Pon un breakpoint en guardar o confirmarEliminacion para inspeccionar datos y flags antes de enviar.

Problemas habituales y correcciones

Síntoma	Causa probable	Qué comprobar o corregir
No se puede contactar la API.	Servidor detenido, URL incorrecta o bloqueo CORS.	API en terminal A; puerto y API_URL coinciden; revisa consola y Network.
GET devuelve 404 para toda la colección.	La propiedad del JSON no se llama estudiantes.	El nombre de la propiedad y la ruta del servicio deben coincidir.
Error de proveedor HttpClient.	Configuración HTTP ausente o arranque con otra configuración.	main.ts usa appConfig y esta contiene provideHttpClient().
No se produce el POST.	No hay suscripción o el formulario es inválido.	Comprueba subscribe, estado del formulario y mensajes de validación.
POST se repite.	Dos suscripciones o guardados simultáneos.	Un único subscribe por operación y guardando como bloqueo en método y botón.
El id no coincide con el número esperado.	Se esperaba un contador local.	Usa el id de cadena devuelto; no asumas una secuencia.
El cero se rechaza como vacío.	Validación basada en falsedad.	Distingue null de 0; conserva los validadores y el estrechamiento de tipos.
La lista parece vacía ante un fallo.	El error se sustituyó por [] sin informar.	Mantén el estado error separado de un 200 con arreglo vacío.
La carga no termina.	El flag solo se limpia en next.	Usa finalize para éxito, error y cancelación.
Se pierden datos al reiniciar.	Se ejecutó api:reset o se sirvió seed.json.	Sirve db.json y restaura la semilla solo de forma explícita.
Comando --delay rechazado.	Se copió un ejemplo de la línea anterior.	Usa la versión fijada y limitación de velocidad en DevTools.
Las pruebas intentan contactar el servidor.	Proveedores HTTP de prueba incorrectos.	Declara provideHttpClientTesting después de provideHttpClient.
Errores en plantillas o imports.	Archivo incompleto o nombre de exportación diferente.	Copia todos los bloques; revisa rutas, mayúsculas y nombre de cada clase.

El simulador permite escribir datos que incumplen las reglas si se llama directamente a la API. La validación de Angular mejora la experiencia y evita envíos accidentales desde el portal; no protege el servidor. En una aplicación profesional se necesitan validación del lado servidor, autorización y una persistencia adecuada, según el caso.

16 Reto de aplicación individual

Reto obligatorio

Añade a Estudiantes un filtro local por nombre y programa después de recibir el listado. Deriva el resultado con computed; conserva la lista original y sigue cada tarjeta por id. El filtrado no debe producir otro GET cada vez que escribes. Distingue “No hay estudiantes registrados” de “No hay coincidencias con este filtro”.

6. Añade una etiqueta accesible a cada control y un botón para limpiar los filtros.
7. Elimina un registro visible con el filtro activo y comprueba que desaparece tras el éxito de DELETE.
8. Mantén carga, error, confirmación y selección al incorporar el nuevo estado.
9. Incluye dos pruebas: filtro sin coincidencias y recuperación de todas las tarjetas al limpiar. Explica qué función de transformación estás comprobando.

Ampliación opcional

Valida en ejecución la respuesta de listar antes de aceptarla como Estudiante[]. Recibe unknown y comprueba arreglo, propiedades, id de cadena, programa admitido y límites numéricos. Rechaza una respuesta mal formada con un mensaje explícito; no la conviertas silenciosamente en una lista vacía. Diseña casos para un registro válido, un id numérico y un promedio fuera de rango.

Justifica por qué el genérico de HttpClient no reemplaza esa validación. Mantén esta ampliación separada del código base para poder comparar ambos comportamientos y conserva la URL de la API local. No necesitas incorporar una nueva dependencia para completar el reto.

17 Preguntas de comprobación

Pregunta	Orientación para comprobar tu respuesta
1 Por qué api.listar no envía HTTP por sí solo	El Observable de HttpClient se ejecuta al suscribirse.
2 Qué ocurre con dos suscripciones al mismo GET	Se inician dos solicitudes; no se comparte automáticamente la respuesta.
3 Por qué el servicio devuelve Observable	Permite a la página decidir la suscripción y manejar estado, éxito y error.
4 Qué distingue un 200 vacío de un error 500	El primero es una consulta correcta sin datos; el segundo es una consulta fallida.
5 Por qué se usa finalize	Limpia estados de actividad en finalización, error o cierre de la suscripción.
6 Por qué no se fabrica el id en el formulario	La API lo asigna; JSON Server 1 usa cadenas y no garantiza una secuencia.
7 Qué distingue PATCH de PUT en esta API	PATCH conserva campos omitidos; PUT reemplaza la representación editable.
8 Qué verifica realmente get<Estudiante[]>	El tipo ayuda a compilar; no comprueba el JSON en ejecución.

Pregunta	Orientación para comprobar tu respuesta
9 Por qué un fallo de POST no debe repetirse a ciegas	El servidor pudo aplicarlo antes de perderse la respuesta; verifica el listado.
10 Qué logra takeUntilDestroyed y qué no garantiza	Cierra la suscripción al destruir la página; no deshace un cambio ya aplicado.
11 Por qué cero y null deben tratarse distinto	Cero es un dato válido; null representa un campo numérico sin completar.
12 Qué demuestra el reinicio de la API sin reset	Los datos se conservaron en db.json y no solo en memoria del navegador.

Explica al menos cuatro respuestas con una solicitud o una prueba del proyecto. Relacionar el concepto con una evidencia concreta te permitirá reconocer el mismo comportamiento en otras APIs y aplicaciones Angular.

19 Referencias oficiales

R1 MDN Métodos de solicitud HTTP

R2 MDN Intercambio de recursos de origen cruzado CORS

R3 JSON Server Documentación oficial y notas de la línea 1

R4 Angular Inyección de dependencias

R5 Angular Configuración de HttpClient

R6 Angular Realización de solicitudes HTTP

R7 Angular API takeUntilDestroyed

R8 Angular Compatibilidad de versiones

R9 Angular Validación de formularios

R10 Angular Pruebas de solicitudes HTTP

R11 Angular Pruebas con Vitest

R12 RxJS API finalize

R13 RxJS API switchMap

R14 Node.js Versiones oficiales